TEMA 2 – Spark Structured Streaming

Se proponen diferentes ejercicios de ingesta y procesamiento de datos en streaming aplicados a un escenario de analítica deportiva. Para ello se utilizará **Apache Spark Structured Streaming** para consumir y procesar en tiempo real los **eventos publicados** previamente en **Apache Kafka**. Se trabajará sobre el topic definido en la entrega anterior, asumiendo que el producer de Kafka ya está implementado. De este modo, el flujo de datos se genera en tiempo real a partir del **producer implementado en la entrega anterior**.

Para el desarrollo de la práctica se proporciona un entorno con Kafka y un Jupyter Notebook con PySpark mediante el fichero *docker-compose.yml,* así como un notebook base *spark_streaming.ipynb* con celdas marcadas para completar. El entorno levanta los servicios en local y monta el directorio de trabajo en el contenedor de Jupyter. Para simplificar la conectividad entre servicios, los contenedores comparten la red de la máquina anfitriona.

Ejercicio 1. Ingesta de eventos desde Kafka

Se pide implementar en PySpark un **stream de lectura desde Kafka** que consuma los eventos publicados en la práctica anterior.

La creación de la sesión de Spark necesaria para trabajar con Structured Streaming y el conector de Kafka ya se proporcionan dentro del notebook adjunto.

Apartado 1.1. Definición del esquema del mensaje

Define el esquema de Spark correspondiente al contenido de los mensajes publicados en Kafka. El **esquema debe incluir todos los campos** de metadatos y de medición de los eventos consumidos.

Apartado 1.2. Lectura del stream de eventos

Completa la constante *KAFKA_TOPIC* con el **nombre del topic definido** en la entrega anterior de Apache Kafka. Se debe reutilizar exactamente el mismo topic para consumir los eventos ya publicados.

Ejercicio 2. Procesamiento en streaming

Se pide transformar los mensajes leídos desde Kafka y **calcular métricas agregadas por jugador** mediante ventanas temporales en streaming. Se requiere usar Spark Structured Streaming.

Apartado 2.1. Parseo del mensaje

Implementa la transformación del stream para deserializar el contenido de los mensajes recibidos desde Kafka utilizando el esquema definido en el ejercicio anterior.

Debe tenerse en cuenta que el contenido de los mensajes en Kafka se recibe en bytes, por lo que será necesario **convertir su contenido a texto antes de aplicar el parseo del mensaje**. Para ello, puede utilizarse una transformación como *selectExpr('CAST(value AS STRING) AS json_str')*.

Apartado 2.2. Agregaciones por jugador en ventanas temporales

Implementa un flujo de agregación en streaming por jugador utilizando ventanas temporales deslizantes y watermark. La agregación debe agrupar, como mínimo, por **ventana temporal, partido y jugador**.

Se debe usar un **watermark de 2 minutos** para tolerar la llegada de eventos tardíos, así como **ventanas deslizantes de 1 minuto con deslizamiento de 30 segundos**.

Tras ello, se deben calcular las siguientes métricas por grupo, redondeando a dos decimales las métricas necesarias:

- Número de eventos
- Frecuencia cardiaca media
- Desviación estándar de la frecuencia cardiaca
- Frecuencia cardiaca máxima

Apartado 2.3. Salida del stream por consola

Completa la escritura del stream para mostrar los resultados en streaming utilizando el **modo *update***. Para esta práctica no es necesario configurar checkpointing, ya que la salida se debe realizar **por consola** con fines de depuración y validación. La salida deberá permanecer activa mientras se estén publicando eventos desde Kafka.

Para facilitar la visualización de los resultados completos en consola, se recomienda desactivar el truncado de columnas mediante *option('truncate', 'false')* y consultar los logs del contenedor de Jupyter con el comando: *docker compose logs -f jupyter.*

Entregables

Se deberá entregar un único archivo comprimido en formato .zip en el Aula Virtual con los siguientes archivos:

- **spark_streaming.ipynb**: Notebook completado con todas las celdas / TODOs resueltas.

Debe incluirse una **memoria explicativa** en la que se describan el esquema del mensaje utilizado para deserializar el JSON y las transformaciones aplicadas sobre el stream. Asimismo, se deberá explicar la consulta en Spark Structured Streaming implementada. Se deben especificar los requisitos del entorno y una guía clara de instalación y ejecución. Esta memoria podrá entregarse como un documento único junto con la práctica anterior de Kafka, o bien como una memoria independiente para esta continuación.